CrowdVision: Custom CNN-Based Face Detection, Crowd Counting and Face Recognition

A Software Engineering Approach

Mini Project Report submitted in the partial fulfilment of

Bachelor of Technology (B.Tech)

in

Computer Engineering

Course: Software Engineering

by

Gauransh Bhartia (B134)

Garv Agrawal (B156)

Adarsh Yadav (B189)

Under the supervision of

Prof. Prashasti Kanikar

(Department of Computer Engineering, MPSTME)



MUKESH PATEL SCHOOL OF TECHNOLOGY MANAGEMENT & ENGINEERING

SVKM's NMIMS University

(Deemed-to-be University)

MUKESH PATEL SCHOOL OF TECHNOLOGY MANAGEMENT & ENGINEERING (MPSTME)

Vile Parle (W), Mumbai-56

Academic Year 2026–27

INDEX

Sr. No.	Contents	Page No.
1	Project Introduction and Authors	3
2	Problem Statement, Objectives and Scope	4
3	Software Requirements Specification (SRS)	5
4	Software Process Model, Estimation and Risks	6
5	System Architecture and Design Diagrams	7
6	Implementation	10
7	Testing and Results	11
8	Screenshots	12
9	Conclusion and Future Scope	14
10	References	14

List of figures

Fig. 1 System architecture (4 layers); Fig. 2 Use case diagram; Fig. 3 Data flow diagram (Level 1); Fig. 4 Class diagram; Fig. 5 Sequence diagram (one frame); Fig. 6 Web console: live detection; Fig. 7 Web console: face recognition; Fig. 8 Web console: session dashboard; Fig. 9 Recognition result; Fig. 10 Evaluation scenes; Fig. 11 Automated test run (17/17 passed).

1. Project Introduction and Authors

1.1 Introduction

Counting people and knowing who is present are everyday needs in classrooms, labs, events and offices. Doing it by hand is slow and error-prone, and most automatic tools depend on large ready-made models (such as YOLO) or cloud services.

CrowdVision is a software product that watches a camera, finds every face, counts the people present, raises an alert when a room is over capacity and recognises enrolled people by name. Both neural networks inside it, CrowdVisionNet (detection, 122,774 parameters) and RecogNet (recognition), were **designed and trained from scratch by the team**, with no YOLO or pretrained weights. It runs in real time on an ordinary laptop CPU, through a desktop app and a browser-based web console.

This report presents CrowdVision from a **Software Engineering** point of view. The CNN is one module of a larger system that was built through the full SDLC: requirements (SRS), an iterative process model, effort estimation, risk management, layered design with UML diagrams, modular implementation, testing at four levels, a defect log and maintenance planning.

1.2 Authors

Sr. No.	Name	Roll No.	Program
1	Gauransh Bhartia	B134	B.Tech Computer Engineering
2	Garv Agrawal	B156	B.Tech Computer Engineering
3	Adarsh Yadav	B189	B.Tech Computer Engineering

Project mentor: Prof. Prashasti Kanikar, Department of Computer Engineering, MPSTME, NMIMS.

1.3 Tools and technologies

Area	Used
Languages	Python 3.10+ (backend, models); HTML, CSS, JavaScript (web console)
Libraries	PyTorch (basic layers only), OpenCV (camera and drawing only), NumPy, Pillow
Datasets	LFW (13,233 face photos), Caltech-101 (7,807 non-face photos)
SE tools	Git and GitHub (version control), pytest (automated testing), VS Code (IDE), draw.io-style UML diagrams

2. Problem Statement, Objectives and Scope

2.1 Problem statement

To design, build and test a software system that, from a live camera, video or photo, **detects every visible face, counts the people in real time, warns when a capacity is exceeded and identifies enrolled people**, using CNNs the team trains from scratch, running offline on a normal laptop CPU, and developed with a disciplined software engineering process.

Our first prototype showed why process matters: trained on 21 images it reached 100% training accuracy but, on test scenes, only F1 67.9% with an average count error of 2.18 people. Measurable requirements, iterative testing and defect tracking turned it into a reliable product.

2.2 Objectives

- Write an **SRS** with 12 functional and 10 measurable non-functional requirements (IEEE 830 style).
- Follow an **iterative and incremental process model**, testing at the end of every iteration.
- Design a **layered, modular architecture** and document it with use case, DFD, class and sequence diagrams.
- Build a fully convolutional CNN that meets **precision and recall ≥ 90%** and a count error ≤ 0.5 people per image.
- Deliver a desktop app and a web console with counting, capacity alert, zone, logs, enrolment and recognition.
- **Verify and validate** the system with unit, integration, system and acceptance tests and keep a **defect log**.
- Plan the project with **COCOMO** effort estimation and a **risk register**.

2.3 Scope

In scope	Out of scope
Face detection, counting and recognition from webcam, video or photo	People whose faces are not visible
Real-time CPU use on a laptop (Windows / Linux)	GPU servers, mobile apps, cloud hosting
Capacity alert, zone, screenshots, recording, CSV logs, attendance	Very dense crowds of hundreds of people
Small groups such as a class or lab	Large identity databases or surveillance
Local web console bound to the same machine	User accounts or storing faces online

Stakeholders: the operator (faculty, lab or event staff) who runs the system; the people being counted or recognised; and the developers who maintain it.

Limitations: the detector is trained mainly on frontal faces, so strong side profiles or faces under about 30 px can be missed; the evaluation scenes are synthetic composites.

3. Software Requirements Specification (SRS)

Requirements were gathered from the problem statement, mentor discussions and testing of the first prototype. Each has an ID and, where possible, a measurable target, so it can be verified by a test (Section 7).

3.1 Functional requirements

ID	Requirement	Priority
FR-01	Read frames from a webcam, a video file or an image.	High
FR-02	Detect every visible face and draw one box per face.	High
FR-03	Display the current people count, peak count and frames per second.	High
FR-04	Raise a visible alert when the count exceeds a user-set capacity.	High
FR-05	Let the user draw a zone and show the count inside it.	Medium
FR-06	Log the count with a timestamp and export it as CSV.	Medium
FR-07	Save screenshots and record video.	Medium
FR-08	Enrol a person with the webcam (about 60 photos).	High
FR-09	Train the recognition model from the interface.	High
FR-10	Show the name of each enrolled person, or "Unknown".	High
FR-11	Record each person's first-seen and last-seen times (attendance).	Medium
FR-12	Change the confidence threshold and capacity while running.	Medium

3.2 Non-functional requirements

ID	Type	Requirement	Measurable target
NFR-01	Accuracy	Detected boxes should be real faces	Precision ≥ 90% on unseen scenes
NFR-02	Accuracy	Real faces should be found	Recall ≥ 90% on unseen scenes
NFR-03	Accuracy	Counts close to the truth	Count error ≤ 0.5 people per image
NFR-04	Performance	Live video on a laptop CPU	≥ 5 frames per second at 640 px
NFR-05	Generalisation	Works on people never seen in training	Test split by person
NFR-06	Portability	Windows and Linux, Python 3.10+	One pip install -r requirements.txt
NFR-07	Usability	First-time user can start quickly	One command; buttons and on-screen help
NFR-08	Maintainability	Modules change independently	Layered design + automated tests
NFR-09	Privacy	No images leave the computer	Offline; server bound to localhost
NFR-10	Independence	No YOLO or pretrained weights	All weights trained from scratch

4. Software Process Model, Estimation and Risks

4.1 Process model: iterative and incremental

We chose an **iterative and incremental model**, close to Boehm's spiral model, instead of waterfall. The behaviour of an ML component cannot be fully specified in advance; it must be measured and improved. Each iteration ended with testing, and the defects found defined the next iteration.

Iteration	What we built	Testing showed	Next step
1. Prototype	Sliding-window CNN, 21 images	100% train, 75% validation; duplicate boxes; slow	New dataset pipeline
2. Data + model	166k crops, fully convolutional CNN	99.87% crop accuracy; many false boxes	Hard-negative mining
3. Robust detection	Mining, edge-face training, cluster merging	Precision 57% → 96%, recall 83% → 99%	Tracker + recognition
4. Product	Tracker, RecogNet, desktop + web UI, tests	17/17 tests pass; all accuracy NFRs met	Maintenance

4.2 Effort estimation (Basic COCOMO, organic mode)

Size: about **2.88 KLOC** (1,955 lines of Python, 840 lines of web code, 89 lines of tests). Organic mode is used because the team is small and the requirements are flexible.

- Effort E = 2.4 × (KLOC)^1.05 = 2.4 × 2.884^1.05 ≈ **7.3 person-months**
- Development time D = 2.5 × E^0.38 = 2.5 × 7.3^0.38 ≈ **5.3 months**
- Average staff P = E / D = 7.3 / 5.3 ≈ **1.4 people**

COCOMO assumes all code is written by hand. We reused libraries (PyTorch, OpenCV), so this is an upper bound.

4.3 Risk management

Risk	Prob.	Impact	Mitigation	Status
Too little training data; model overfits	High	High	Public datasets, augmentation, split by person	Resolved
Duplicate or wrong boxes inflate the count	High	High	Box regression, cluster merging, nested-box rule, tracker	Resolved
Too slow on a CPU	Med	High	Fully convolutional design, --width option	Resolved
False detections in the demo room	Med	Med	Hard-negative mining, collect_backgrounds.py	Mitigated
Privacy of face data	Med	High	Offline, localhost only; faces excluded from Git	Mitigated
Library / OpenCV GUI incompatibility	Med	Low	Pinned requirements; documented fix	Resolved

5. System Architecture and Design Diagrams

5.1 Layered architecture

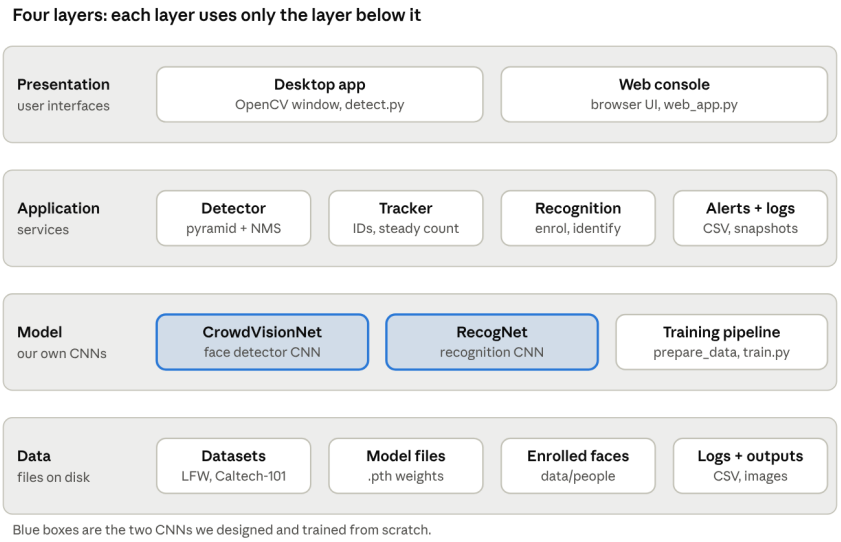


Fig. 1: System architecture: Presentation, Application, Model and Data layers

- **Presentation:** desktop OpenCV window (`detect.py`, `face_recognition_app.py`) and web console (`web/index.html`).
- **Application:** Engine (`web_app.py`), IoU tracker, alerts, counting zone, logging.
- **Model:** CrowdVisionNet (`model.py`, `detector.py`) and RecogNet.
- **Data:** datasets, trained weights (.pth), enrolled faces, CSV logs.

5.2 Design principles applied

- **Modularity:** 10 modules, each with one job.
- **Low coupling:** the interface only calls `detect(frame)` and receives boxes; it does not know how the CNN works. The web console was added without changing the model.
- **High cohesion:** all box-merging logic lives in `detector.py`, so the biggest accuracy fix (defect D-03) touched one file.
- **Separation of concerns:** data preparation, training, inference and UI are separate scripts, which avoids hidden technical debt in ML systems.

CNN design (CrowdVisionNet): 48×48 input → Conv 16 → Conv 32 → Conv 64 (each 3×3 + BatchNorm + ReLU + MaxPool) → Conv 4×4, 96 filters → two 1×1 heads (face/non-face, box offsets). With no Linear layers it scores a whole frame in one pass. Detection pipeline: image pyramid → CNN → threshold 0.99 → box refinement → cluster merge with voting → nested-box rule → tracker (median count over 15 frames).

5.3 Use case diagram

The **Operator** starts detection, views the count, sets capacity, draws a zone, enrols people, trains the recogniser and exports logs. The **Enrolled person** takes part in enrolment and recognition.

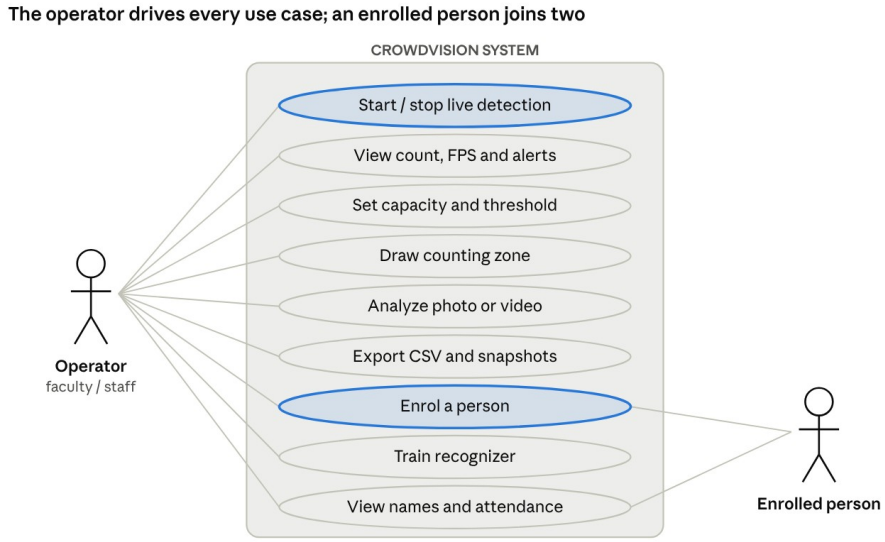


Fig. 2: Use case diagram

5.4 Data flow diagram (Level 1)

Frames flow from the camera through capture (1.0), detection (2.0), tracking (3.0), recognition (4.0) and display (5.0); enrolment and training (6.0) write to the data stores D1 models, D2 enrolled faces and D3 logs.

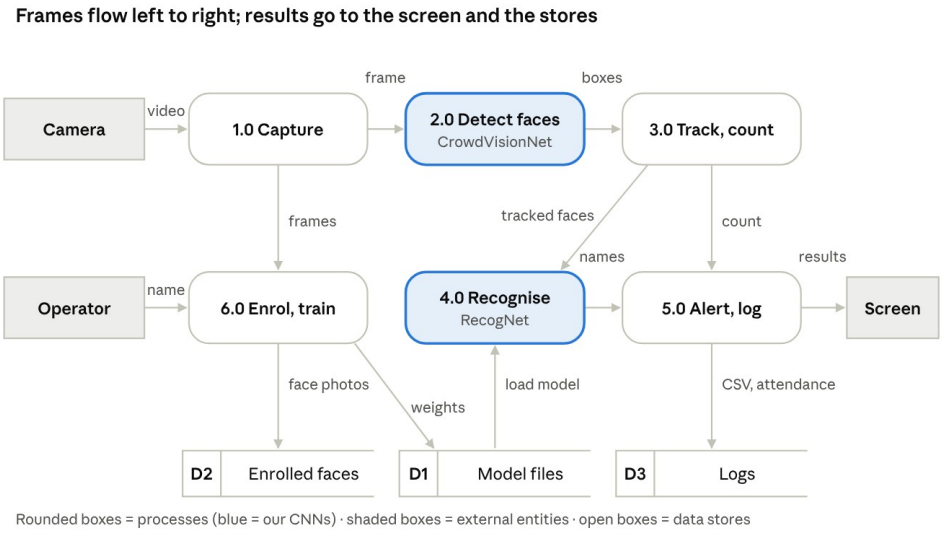


Fig. 3: Data flow diagram, Level 1

5.5 Class diagram

Engine coordinates the system. FaceDetector **owns** CrowdVisionNet and Recognizer owns RecogNet (composition); FaceTracker keeps one Track per person; the web Handler calls Engine.

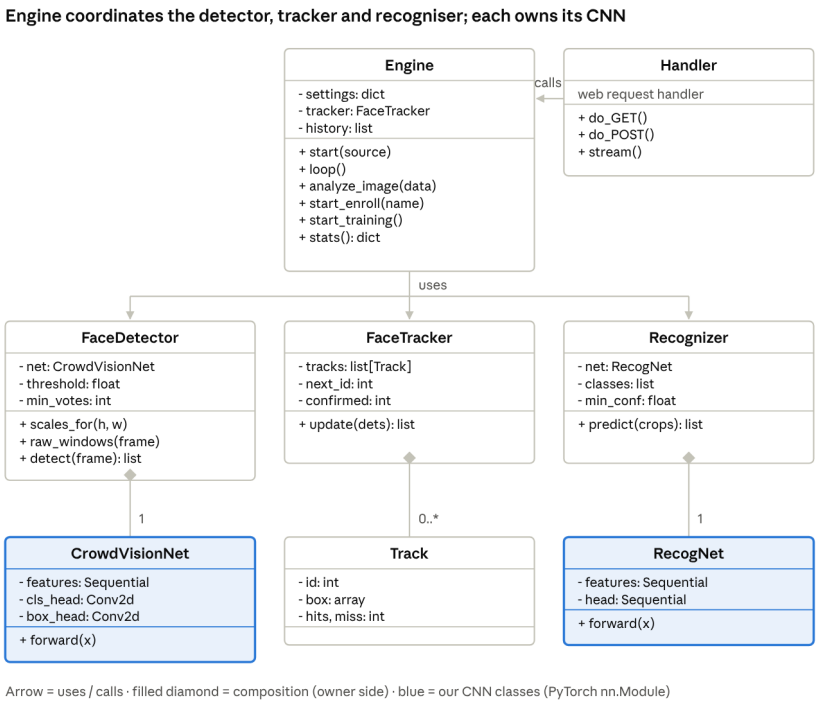


Fig. 4: Class diagram

5.6 Sequence diagram

For each frame, Engine reads the camera, calls detect(frame), updates the tracker, calls predict for each face, draws the result and checks the capacity alert.

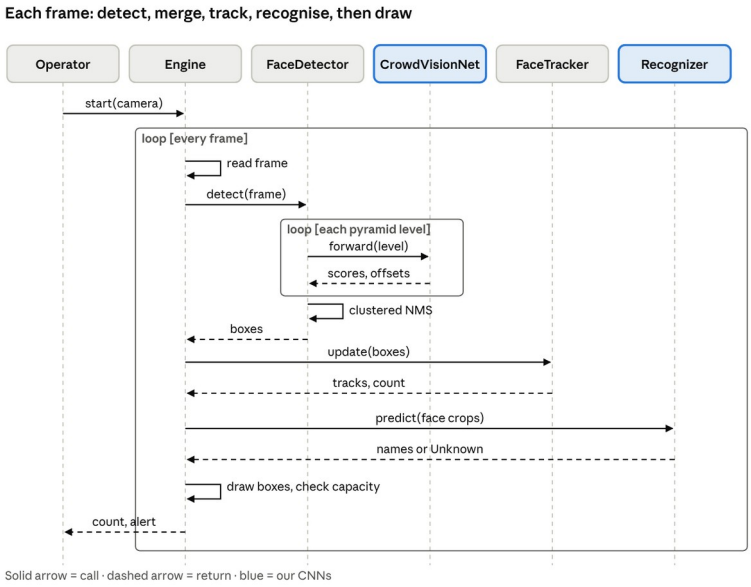


Fig. 5: Sequence diagram for processing one frame

6. Implementation

Module	Responsibility	Requirements
model.py	CrowdVisionNet architecture	FR-02, NFR-10
download_datasets.py, prepare_data.py	Download data; build ~166k crops, split by person	NFR-05
train.py	Training, hard-negative mining, metrics, plots	NFR-01, 02
detector.py	Pyramid inference, box refinement, cluster merge	FR-02, NFR-01..04
detect.py	Desktop app: tracker, zone, alert, screenshots, recording, CSV	FR-01..07, 12
face_recognition_app.py	Enrol, train RecogNet, recognise, attendance	FR-08..11
web_app.py, web/index.html	Web console: live feed, analyser, recognition, dashboard	FR-01..12, NFR-07, 09
evaluate.py	System test on 300 unseen scenes	NFR-01..03
tests/test_crowdvision.py	17 automated unit and integration tests	NFR-08
requirements.txt, README.md, .gitignore	Configuration, setup guide, privacy rules for Git	NFR-06, 07, 09

6.1 Version control and configuration

The code is kept in **Git and GitHub**. `requirements.txt` pins the dependencies (NFR-06) and `.gitignore` keeps datasets and enrolled face photos out of the repository (NFR-09). Data preparation uses a fixed random seed, so every build of the training set is reproducible (checked by test UT-12).

6.2 How to run

```
pip install -r requirements.txt
python detect.py                 # desktop live demo
python face_recognition_app.py   # enrol / train / recognise
python web_app.py                # web console
python -m pytest tests -v        # 17 automated tests
```

6.3 Maintenance plan

Type	In CrowdVision
Corrective	Each bug from the defect log becomes a new automated test before it is fixed
Adaptive	Retrain for a new room with collect_backgrounds.py; adapt to new library versions
Perfective	New features added without touching the model (web console, recognition)
Preventive	Run the automated tests after every change (regression testing)

7. Testing and Results

Testing was done at four levels, each with a different purpose: **unit** and **integration** tests (white-box, automated with pytest), **system** testing against the NFRs, and **acceptance** testing against the FRs (black-box).

7.1 Unit and integration tests (17 of 17 pass)

ID	What is tested	Expected	ID	What is tested	Expected
UT-01	CNN output, one crop	2 + 4 values	UT-10	Tracker keeps ID	Same ID
UT-02	CNN output, full frame	55×75 map	UT-11	Crowd levels	LOW/HIGH/OVER
UT-03	Parameter count	122,774	UT-12	Split is reproducible	Same split
UT-04	Frame = sliding window	Equal	UT-13	Face crop at edge	No error
UT-05	IoU values	1 / 0 / ⅓	IT-01	Blank frame	0 faces
UT-06	NMS removes duplicate	2 of 3 kept	IT-02	Unseen person photo	1 face found
UT-07	Cluster merge votes	1 box, 2 votes	IT-03	Trained model loads	Ready
UT-08	Nested-box rule	Small box kept	IT-04	Recogniser output	Valid name
UT-09	Tracker needs 2 hits	Shown at frame 2			

7.2 System testing: NFR verification

Requirement	Target	Measured (300 unseen scenes, 1,438 faces)	Result
NFR-01 Precision	≥ 90%	96.1% (first prototype: 57.3%)	Met
NFR-02 Recall	≥ 90%	99.0% (first prototype: 83.3%)	Met
NFR-03 Count error	≤ 0.5	0.23 people per image; 82.3% of images exact	Met
NFR-04 Speed	≥ 5 FPS	about 77 ms per frame (≈ 13 FPS), 2-core CPU, 60 px min face	Met
NFR-05 Generalisation	Unseen people	99.87% crop accuracy (P 99.87%, R 99.48%, F1 99.67%)	Met
NFR-09 / 10	Offline; from scratch	Server on 127.0.0.1; all weights trained by us	Met

7.3 Acceptance testing

TC	Req.	Test	Result
TC-01/02	FR-01, 02	Webcam and video file show one box per face	Pass
TC-03/04	FR-03	Dashboard updates; photo analyser returns count	Pass
TC-05	FR-07	Save snapshot	Pass
TC-06/07	FR-09, 10	Train recogniser; names or "Unknown" shown	Pass
TC-08 to 12	FR-04, 05, 06, 08, 11, 12	Capacity alert, zone, CSV export, enrolment, threshold slider	To verify in live demo

7.4 Defect log

ID	Defect	Found by	Fix
D-01	100% train vs 75% validation accuracy	Validation test	New 166k-crop dataset, augmentation
D-02	Several boxes on one face	Live test	Box regression, clustered NMS
D-03	Big stretched boxes; precision 57%	System test	Size-aware merging + nested-box rule
D-04	Face at frame edge not detected	Manual test	Frame padding + edge-cut training
D-05	One person counted as 61 unique faces	Acceptance test	Count only after 2 consecutive hits
D-06	Fine-tuning ran out of memory	Training run	Cap at 40,000 hard negatives; --resume
D-07	Window failed to open on Windows	Acceptance test	Reinstall GUI OpenCV; documented in README

8. Screenshots

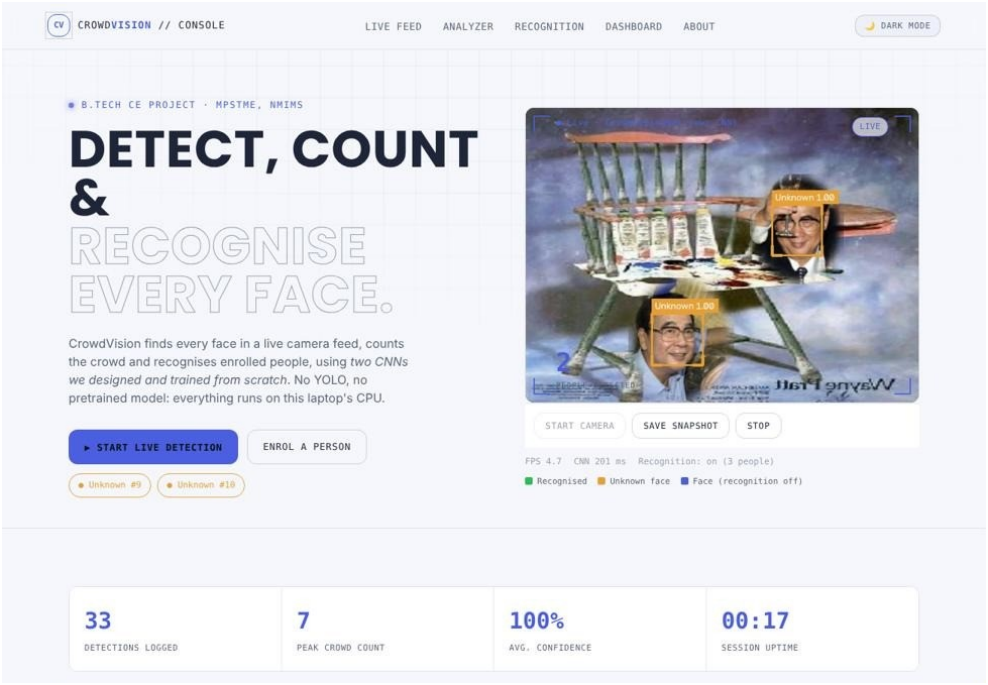


Fig. 6: Web console: live detection with face count, FPS and session statistics

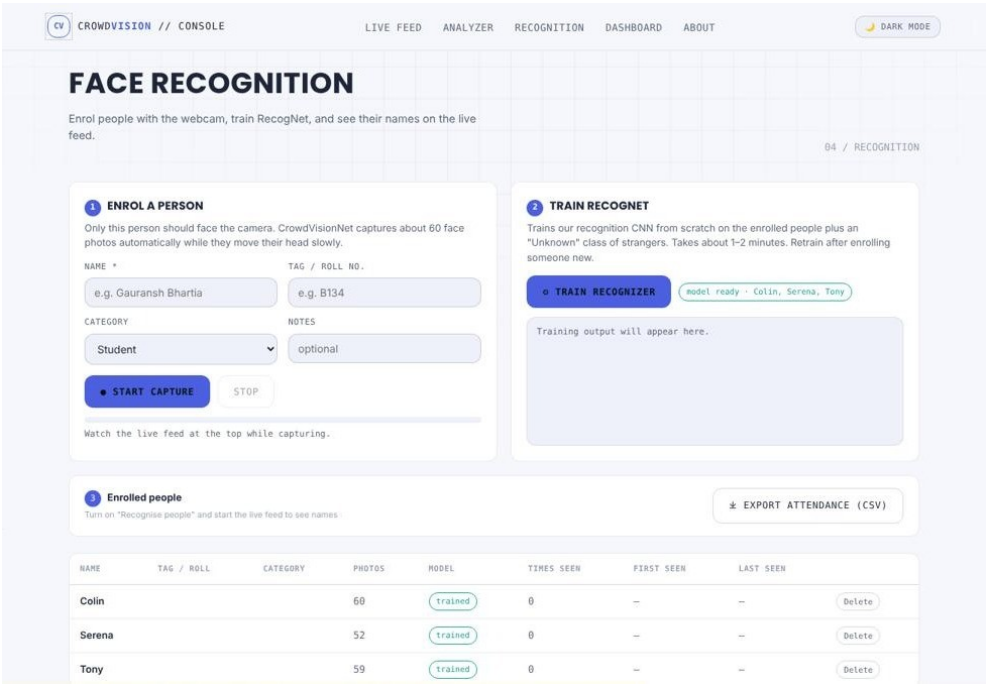


Fig. 7: Web console: enrol a person, train RecogNet and list enrolled people

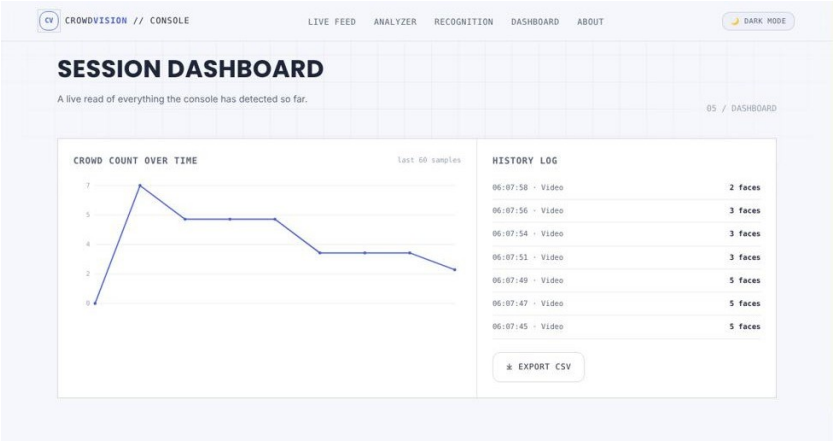


Fig. 8: Web console: session dashboard with crowd count over time and history log

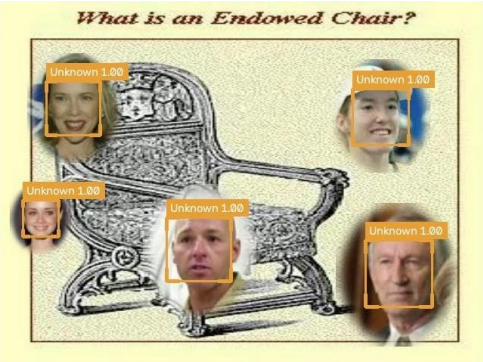


Fig. 9: Recognition result: people who are not enrolled are labelled "Unknown"



Fig. 10: System test on unseen scenes (blue = ground truth, green = prediction)

```
$ python -m pytest tests -v
platform linux -- Python 3.13.16, pytest-9.1.1, pluggy-1.7.0 -- /usr/bin/python
plugins: platformdirs-4.12.3, anyio-4.15.1
collecting ... collected 17 items
tests/test_crowdvision.py::test_ut01_crop_output_shape PASSED            [  5%]
tests/test_crowdvision.py::test_ut02_fully_convolutional_frame_output PASSED [ 11%]
tests/test_crowdvision.py::test_ut03_parameter_count PASSED              [ 17%]
tests/test_crowdvision.py::test_ut04_full_frame_equals_sliding_window PASSED [ 23%]
tests/test_crowdvision.py::test_ut05_iou_values PASSED                   [ 29%]
tests/test_crowdvision.py::test_ut06_nms_removes_duplicates PASSED       [ 35%]
tests/test_crowdvision.py::test_ut07_cluster_merge_votes PASSED          [ 41%]
tests/test_crowdvision.py::test_ut08_nested_box_rule PASSED              [ 47%]
tests/test_crowdvision.py::test_ut09_tracker_needs_two_hits PASSED       [ 52%]
tests/test_crowdvision.py::test_ut10_tracker_keeps_id PASSED             [ 58%]
tests/test_crowdvision.py::test_ut11_crowd_levels PASSED                 [ 64%]
tests/test_crowdvision.py::test_ut12_person_split_is_deterministic PASSED [ 70%]
tests/test_crowdvision.py::test_ut13_face_crop_at_frame_edge PASSED      [ 76%]
tests/test_crowdvision.py::test_it01_blank_frame_has_no_faces PASSED     [ 82%]
tests/test_crowdvision.py::test_it02_detects_face_in_unseen_photo PASSED [ 88%]
tests/test_crowdvision.py::test_it03_trained_model_loads_and_runs PASSED [ 94%]
tests/test_crowdvision.py::test_it04_recognizer_returns_valid_names PASSED [100%]
============================== 17 passed in 1.92s ==============================
```

Fig. 11: Automated test run: 17 of 17 tests passed (pytest)

9. Conclusion and Future Scope

CrowdVision meets all its accuracy and performance requirements: **96.1% precision, 99.0% recall and a count error of 0.23 people per image**, in real time on a laptop CPU, using CNNs designed and trained from scratch. The main lesson is that the biggest quality gains came from software engineering, not from a bigger model. Measurable NFRs made progress visible, testing at several levels found different defects (system testing found D-03, acceptance testing found D-05 and D-07), and the low-coupling design meant the largest fix touched only one module. The iterative process took F1 from 67.9% to **97.5%.**

Future scope: train on more side-profile and low-light faces; add density-map counting for very dense crowds; export the model to ONNX for a Raspberry Pi or a phone; add multi-camera support with a central dashboard; set up continuous integration so the tests run on every GitHub push.

10. References

[1] W. W. Royce, "Managing the Development of Large Software Systems," Proc. IEEE WESCON, 1970, pp. 1–9.

[2] B. W. Boehm, "A Spiral Model of Software Development and Enhancement," IEEE Computer, vol. 21, no. 5, pp. 61–72, 1988.

[3] B. W. Boehm, Software Engineering Economics. Englewood Cliffs, NJ: Prentice-Hall, 1981.

[4] IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications, IEEE, 1998.

[5] D. Sculley et al., "Hidden Technical Debt in Machine Learning Systems," Advances in NeurIPS 28, 2015, pp. 2503–2511.

[6] S. Amershi et al., "Software Engineering for Machine Learning: A Case Study," Proc. IEEE/ACM ICSE-SEIP, 2019, pp. 291–300.

[7] E. Breck, S. Cai, E. Nielsen, M. Salib and D. Sculley, "The ML Test Score: A Rubric for ML Production Readiness and Technical Debt Reduction," Proc. IEEE Big Data, 2017, pp. 1123–1132.

[8] J. M. Zhang, M. Harman, L. Ma and Y. Liu, "Machine Learning Testing: Survey, Landscapes and Horizons," IEEE Trans. Software Engineering, vol. 48, no. 1, pp. 1–36, 2022.

[9] A. Bochkovskiy, C.-Y. Wang and H.-Y. M. Liao, "YOLOv4: Optimal Speed and Accuracy of Object Detection," arXiv:2004.10934, 2020.

[10] Y. Li, X. Zhang and D. Chen, "CSRNet: Dilated Convolutional Neural Networks for Understanding the Highly Congested Scenes," Proc. IEEE CVPR, 2018, pp. 1091–1100.

[11] L. Boominathan, S. S. S. Kruthiventi and R. V. Babu, "CrowdNet: A Deep Convolutional Network for Dense Crowd Counting," Proc. ACM Multimedia, 2016, pp. 640–644.

[12] G. B. Huang, M. Ramesh, T. Berg and E. Learned-Miller, "Labeled Faces in the Wild," Univ. of Massachusetts, Amherst, Tech. Rep. 07-49, 2007.